6.2 Работа с файлами в Go

Содержание

1. Работа с дескрипторами файлов
2. Получение дескриптора файла
3. Создание директории и JSON файла
4. Конкурентная запись в файл
5. Работа с временными файлами и директориями
6. Обход директорий
7. Изменение прав доступа

1. Работа с дескрипторами файлов

Дескрипторы файлов — это идентификаторы, которые операционная система присваивает открытым файлам. В Go они скрыты за типом `*os.File`.

Файл: `file.txt`

```
info
```

Файл: `desc.md`

```
### Что такое дескрипторы файлов?

Дескриптор файла — это уникальный идентификатор (целое число),
который операционная система присваивает открытому файлу или ресурсу ввода-вывода.
Дескрипторы используются для управления доступом к файлам и выполнения операций чтения, записи или
закрытия.

- **Как это работает**:
    - Когда программа открывает файл (например, через системный вызов `open`), ОС:
       1. Проверяет права доступа.
       2. Создает запись в системной таблице открытых файлов.
       3. Возвращает процессу дескриптор, который является индексом в таблице дескрипторов
процесса.
    - Каждый процесс имеет свою таблицу дескрипторов, начиная с 0 (stdin), 1 (stdout) и 2
```

```
(stderr).
    - Дескрипторы уникальны в пределах процесса, но разные процессы могут иметь одинаковые номера дескрипторов для разных файлов.

### Дескрипторы в Go

В Go дескрипторы файлов скрыты за типом `*os.File`.
Go взаимодействует с ОС, чтобы получить дескриптор, и оборачивает его в объект `*os.File`.

- **Доступ к дескриптору**:
    - Метод `File.Fd()` возвращает дескриптор файла как `uintptr`.

- **Использование**:
-     Методы `*os.File`, такие как `Read` и `Write`, используют дескриптор для выполнения системных вызовов.
-     Go абстрагирует низкоуровневые детали, делая работу с дескрипторами прозрачной для разработчика.
```

Код: main.go

```
package main

import (
 "fmt"
  "os"
)

func main() {
   file, err := os.Open("file.txt")
   if err != nil {
      fmt.Println("Ошибка чтения файла:", err)
      return
   }

   file.Write([]byte("Hello Go!"))
   file.Write([]byte("Bye Go!"))

   file.Close()

   fileInfo, err := os.Stat("file.txt")
   if err != nil {
      fmt.Println("Ошибка:", err)
      return
   }

   fmt.Println("Имя:", fileInfo.Name())
   fmt.Println("Размер:", fileInfo.Size(), "байт") f
   mt.Println("Время изменения:", fileInfo.ModTime())
   fmt.Println("Права доступа:", fileInfo.Mode())
   fmt.Println("Это директория?", fileInfo.IsDir())
}
```

Объяснение:

- `os.Open` открывает файл, возвращая `*os.File` .
- `os.Stat` получает метаданные файла (имя, размер, время изменения, права доступа).

Замечание: Попытка записи в файл, открытый только для чтения (`os.Open`), вызовет ошибку. Для записи используйте `os.OpenFile` с флагом `os.O_RDWR`.

2. Получение дескриптора файла

Дескриптор файла можно получить с помощью метода `File.Fd()`.

Файл: `file.txt`

```
123
456
```

Код: main.go

```
package main

 import (
 "fmt"
 "os"
)

func main() {
   file, err := os.Open("file.txt")
   if err != nil {
      fmt.Println("Ошибка открытия файла:", err)
      return
   }
   defer file.Close()

   fmt.Println("Дескриптор файла:", file.Fd())
}
```

Объяснение:

`File.Fd()` возвращает дескриптор файла как `uintptr`.

Дескриптор — это низкоуровневый идентификатор, используемый ОС для операций с файлом.

3. Создание директории и JSON файла

Создание директорий и файлов конфигурации — распространенная задача в Go.

Файл: `myapp_config/config.json`

```
{
  "appName": "MyAwesomeApp",
  "version": "1.0.0"
}
```

Код: main.go

```
package main

import (
   "encoding/json" "fmt"
   "os"
)

type Config struct {
   AppName string `json:"appName"`
   Version string `json:"version"`
}

func main() {
   dirName := "myapp_config"
   err := os.Mkdir(dirName, 0755)
   if err != nil {
      fmt.Printf("Ошибка при создании директории: %v\n", err) return
   }
   fmt.Printf("Директория '%s' создана\n", dirName)

   config := Config{
      AppName: "MyAwesomeApp", Version: "1.0.0",
   }

   filePath := fmt.Sprintf("%s/config.json", dirName) file, err
   := os.Create(filePath)
   if err != nil {
      fmt.Printf("Ошибка при создании файла: %v\n", err)
      return
   }
```

```
    defer file.Close()

    encoder := json.NewEncoder(file)
    encoder.SetIndent("", " ")
    if err = encoder.Encode(config); err != nil {
        fmt.Printf("Ошибка при записи JSON: %v\n", err)
        return
    }
    fmt.Printf("Файл конфигурации создан: %s\n", filePath)

    fmt.Println("\nСодержимое директории:")
    files, err := os.ReadDir(dirName)
    if err != nil {
        fmt.Printf("Ошибка при чтении директории: %v\n",
        err)
        return
    }

    for _, file := range files { info,
        _ := file.Info()
        fmt.Printf("- %s (%.1f KB)\n", file.Name(),
        float64(info.Size())/1024)
    }
}
```

Объяснение:

- `os.Mkdir` создаёт директорию с указанными правами доступа (0755).
- `json.NewEncoder` записывает JSON с отступами для читаемости.
- `os.ReadDir` читает содержимое директории, отображая имена файлов и их размеры.

4. Конкурентная запись в файл

Конкурентная запись требует синхронизации для избежания конфликтов.

Код: `main.go`

```
package main

import (
  "fmt"
  "os"
  "sync"
)

func main() {
   var mu sync.Mutex
   file, err := os.Create("concurrent.txt")
   if err != nil {
      fmt.Println("Ошибка:", err)
      return
   }
   defer file.Close()

   mu.Lock()
   file.WriteString("Запись из горутины")
    mu.Unlock()
}
```

Объяснение:

`sync.Mutex` используется для синхронизации доступа к файлу.

В примере показана базовая защита записи, хотя в данном случае она избыточна, так как запись выполняется в одной горутине.

5. Работа с временными файлами и директориями

Временные файлы и директории полезны для тестирования или временного хранения данных.

Код: `main.go`

```
package main

import (
  "fmt"
  "os"
  "path/filepath"
)

func main() {
   tmpFile, err := os.CreateTemp("", "example_*.txt")
   if err != nil {
      fmt.Printf("Ошибка создания временного файла: %v\n", err)
      return
   }
   defer os.Remove(tmpFile.Name())
   defer tmpFile.Close()

   fmt.Printf("Создан временный файл: %s\n", tmpFile.Name())
```

```
    content := []byte("Временные данные для теста\nВторая строка")
    if _, err = tmpFile.Write(content); err != nil {
        fmt.Printf("Ошибка записи во временный файл: %v\n", err) return
    }


    readData, err := os.ReadFile(tmpFile.Name()) if
    err != nil {
        fmt.Printf("Ошибка чтения временного файла: %v\n", err)
        return
    }
    fmt.Printf("\nСодержимое файла:\n%s\n", string(readData))

    tmpDir, err := os.MkdirTemp("", "example_dir_*") if
    err != nil {
        fmt.Printf("Ошибка создания временной директории: %v\n", err)
        return
    }
    defer os.RemoveAll(tmpDir)
    fmt.Printf("\nСоздана временная директория: %s\n", tmpDir) for


    i := 1; i <= 3; i++ {


        filePath := filepath.Join(tmpDir, fmt.Sprintf("file%d.txt", i))
        if err = os.WriteFile(filePath, []byte(fmt.Sprintf("Файл %d", i)), 0644); err != nil {
            fmt.Printf("Ошибка создания файла %s: %v\n", filePath, err)
            return
        }
    }

    files, err := os.ReadDir(tmpDir)
    if err != nil {
        fmt.Printf("Ошибка чтения директории: %v\n", err)
        return
    }

    fmt.Println("\nСодержимое временной директории:") for
    _, file := range files {
        content, _ := os.ReadFile(tmpFile.Name())
        fmt.Printf("- %s (%d bytes)\n", file.Name(), len(content))
    }
}
```

Объяснение:

- `os.CreateTemp` создаёт временный файл с шаблоном имени.
- `os.MkdirTemp` создаёт временную директорию.
- `defer os.Remove` и `os.RemoveAll` очищают временные ресурсы.
- `os.ReadDir` читает содержимое директории.

6. Обход директорий

Обход директорий позволяет рекурсивно перечислять файлы и поддиректории.

Файл: `dir/some_text.txt`

```
asdf
```

Файл: `text.txt`

```
asdad
```

Код: main.go

```
package main

import (
  "fmt"
   "os"
   "path/filepath"
)

  func main() {
   path := "."
   err := filepath.Walk(path, func(path string, info os.FileInfo, err error) error {
   if err != nil {
         fmt.Printf("Ошибка доступа к пути %q: %v\n", path, err)
         return err
      }

      if info.IsDir() {
      return nil
      }

      fmt.Println(path)
      return nil
   })

   if err != nil {
      fmt.Printf("Ошибка обхода директории: %v\n", err)
   }
}
```

Объяснение:

- `filepath.Walk` рекурсивно обходит директорию, вызывая функцию для каждого файла или поддиректории.
- Пропускаются директории (`info.IsDir()`), выводятся только пути к файлам.

7. Изменение прав доступа

Go позволяет изменять права доступа к файлам с помощью `os.Chmod` .

Код: `main.go`

```
package main

  import (
  "fmt"
  "os"
)

func main() {
   err := os.Chmod("img.png", 0642)
   if err != nil {
      fmt.Println("Ошибка:", err)
      return
   }

   fmt.Println("Права доступа изменены на 0644")
}
```

Объяснение:

- `os.Chmod` изменяет права доступа к файлу (например, 0642).
- Права доступа задаются в восьмеричной системе (например, 0644 — чтение/запись для владельца, чтение для остальных).

Итог

Рассмотрели ключевые аспекты работы с файлами и директориями в Go:

- **Дескрипторы**: Используйте `*os.File` и `File.Fd()` для доступа к низкоуровневым дескрипторам.
- **Создание директорий**: Используйте `os.Mkdir` и `os.MkdirTemp` для создания постоянных и временных директорий.
- **Конкурентность**: Применяйте `sync.Mutex` для безопасной записи в файлы. **Временные файлы**: Используйте `os.CreateTemp` и `os.MkdirTemp` для временных данных.
- **Обход директорий**: Используйте `filepath.Walk` для рекурсивного перечисления файлов.
- **Права доступа**: Изменяйте права с помощью `os.Chmod` .

Для дополнительной информации обратитесь к документации Go по пакетам `os` , `path/filepath` , `encoding/json` , и `sync` .